

REACTIVE TRADER

A CLEAN-ARCHITECTURE CASE STUDY · AND THE THINGS IT UNLOCKED

▸ mounting @rtc/domain · rxjs-only	PURE
▸ dependency rule	ENFORCED
▸ contract + visual-diff net · 25 gates	GREEN
▸ clients [react · native · solid*]	2 + 1
▸ jarvis.ai · MCP surface	STANDBY

READY

// 00 · ORIGIN

It started as an experiment.

SPEC-DRIVEN DEVELOPMENT

Take a real, non-trivial app. Generate specs from it. Drop those specs into a blank repo — and ask an AI to rebuild the app from the specs alone, in a deliberately different architecture, never seeing the original. How close does it get?

STAGE A

Generate specs

From Adaptive's original ReactiveTraderCloud — Gherkin features, domain markdown, service contracts. Stack-agnostic: what, never how.

→

STAGE B

Rebuild blind

Specs → blank repo. Build from specs only — no sight of the original. Chosen architecture: clean, ports & adapters, framework-swappable.

→

STAGE C

Reconcile

Three-way audit: original vs spec vs clone. Every divergence traced to where it broke.

// 00 · ORIGIN

Close. Not exact.

~14 BEHAVIOURAL DIVERGENCES AUDITED



IMPLEMENTATION-FIDELITY

Spec captured it correctly — the build satisficed. Physics bubbles shipped static; `USD 12,345` became `+12.3k`; one sort direction flipped.

SPEC-FIDELITY

Concentrated at the invisible backend. Server-authoritative logic and delivery seams were flattened, guessed, or dropped in spec generation.

The spec captured even the subtle behaviours faithfully —

- `draggable d3-force position bubbles`
- `a 3×-iteration new-row flash`
- `server-driven competing-quote auto-reject`

A strong outcome for "generate specs from app A, rebuild as app B."

// 00 · ORIGIN

The specs — not the code — are the portable asset.

THE AMPLIFIER — WHICH CODE DID THE TESTS COME FROM?

The clone's contract & visual tests were derived from the clone's own new code — which had already drifted — not from the original specs. So they asserted whatever the build did and ratified every wrong choice. Green tests, false confidence.

IN THE AI ERA, THE HUMAN'S JOB MOVES UP A LAYER: CHECK THE SPECS. FOUR PRACTICES, ONLY PAYING OFF TOGETHER —

01

Clean architecture

A pure domain core — with RxJS as one deliberate, boundary-only exception.

02

Spec-driven

Behaviour is captured as executable specs first.

03

The right test mix

Contract + visual-diff carry the confidence; each kind covers the others' blind spots.

04

AI-assisted

Tight contracts + fast feedback let an agent move safely.

01

THE DISCIPLINE

Clean Architecture

Pure domain logic. Dumb UI. Dependencies point inward — and the rule is machine-enforced, not aspirational.

// 01 · CLEAN ARCHITECTURE

Four principles that override tech choices.

01 Make choices, defer commitment

Picking a technology is fine; binding the codebase to it is not. Framework types never cross inward boundaries — they're bound at a single composition root.

02 The Dependency Rule

Source-code dependencies point only inward. Entities know nothing of use cases; use cases know nothing of presenters; presenters know nothing of the UI framework.

03 Dumb UI

The UI renders state and emits intents. No business logic, no transport awareness, no orchestration. A React→Solid rewrite stays tractable.

04 Behavioural tests as insurance

Tests describe what, not how. They don't import React, RxJS, or Playwright internals — so they survive technology swaps.

// 01 · CLEAN ARCHITECTURE

Every ring is a package boundary.

SOURCE ARROWS
POINT INWARD →



① ENTITIES
Price · Trade · Rfq · Candle — what a thing is, independent of any app.

② USE CASES + PORTS
12 use cases & the port interfaces they depend on — how this system behaves.

③ INTERFACE ADAPTERS
client-core presenters, WsAdapter, react-bindings, shared DTOs — and simulators (real gateways, not mocks).

④ FRAMEWORKS & DRIVERS
client-react · client-react-native · ws-effects · Vite · the Node server host. The replaceable detail.

"The business truth is the yolk; each shell is a replaceable detail."

// 01 · CLEAN ARCHITECTURE

The rule isn't a convention. It's compiled.

✓ ALLOWED — points inward

client-react ⟶ react-bindings ⟶ client-core

client-core ⟶ shared ⟶ domain

server ⟶ domain · shared · ws-effects

✗ REJECTED — fails CI

domain —✗— shared # domain-stays-pure

ws-effects —✗— domain # ws-effects-stays-pure

ui —✗— rxjs # gate 26

domain —✗— node:* # no-node-builtins

dependency-cruiser

7 rules encode the ring boundaries as a graph. No layer violation merges.

pnpm strict mode

Enforces the domain's single-dependency constraint at install time.

25 grep gates + no-circular

Structural invariants types can't catch — and zero import cycles, anywhere.

// 01 · CLEAN ARCHITECTURE

Pure — with one deliberate exception: RxJS.

PURE*

ALLOWED INSIDE @rtc/domain

Entities · value objects · 12 use cases · port interfaces · in-memory simulators — vanilla TypeScript, plus RxJS as the single boundary stream type.

FORBIDDEN — AND UNREACHABLE

No React. No DOM. No React Native. No node built-ins. No knowledge that a UI or a server even exists.

"BUT ISN'T RXJS A DEPENDENCY?" — YES, ON PURPOSE

Total decoupling from every library is a fantasy if you want their benefits — the same is true of React or Solid at the edge. So we allow a very small, deliberate set of exceptions, and RxJS is the one in the core: an acceptable, boundary-only coupling for reactive streams.

It's a safe bet: RxJS is effectively final — its creator has said no further breaking changes are expected unless the TC39 Observable standard moves, at which point it adapts.

AND STILL REPLACEABLE

Clear layer boundaries + contract coverage mean even this can be regenerated. Just as React → Solid, the stream primitive can move:

```
RxJS ↔ async/await + generators
     ↔ effect-ts
```

// 01 · CLEAN ARCHITECTURE

The life of a price tick.

CONTROL FLOWS OUT
IMPORTS POINT IN



// 01 · MAKE CHOICES, DEFER COMMITMENT

The cost of a wrong decision: near zero.

EVERY TECHNOLOGY
IS A PLUGIN

Picking a library, a framework, a test runner, a third party is genuinely hard — the call is always rushed, under-informed, and a right answer today can turn wrong in two years.

> So we made being wrong cheap. Each technology sits behind a boundary — swap it like a plugin and the rest of the system, and the tests, never notice. *Except when it runs better.*

Less stress per decision — which, paradoxically, lets you decide faster.

COST TO REPLACE — THE REPLACEABILITY MATRIX

UI framework (React → Solid)	~1 week
Stream primitive (RxJS → async/await · effect-ts)	~1 week
State ↔ UI bindings	~1 day
Server host (Node + ws)	~2 days
Build tooling · unit runner	~1 day
E2E driver (per new driver)	~3 days

Each cost holds only because inner layers never import outward, ports are inverted, and tests assert behaviour — not implementation.

02

THE VERIFICATION

The backbone.

AI is non-deterministic. We fight back with hard determinism — lints, gates, contract tests — so most of the time we don't even have to look. It's the one thing that lets us move fast with confidence, not babysit a whack-a-mole of returning bugs.

// 02 · DETERMINISM

Fight non-determinism with determinism.

2ND ONLY TO
THE ARCHITECTURE

AI is non-deterministic — so we surround it with an aggressive deterministic net.

`lints` `25 gates` `type-aware checks` `dependency-cruiser` `contract tests`

When linting and gates enforce what we'd otherwise catch in review — clean architecture, decoupling, separation of concerns, simplicity — there's little left for a human to read. The "dark software factory" stops being sci-fi.

THE INVARIANT

The thing that stays true no matter the tech or the architecture — the app's core behaviour.

Behavioural specs encode the invariants. They don't import React, RxJS, or a driver — so they survive every swap. That's what we lean on.

// 02 · VERIFICATION

Tests are layered like the system.

NO TEST IMPORTS A TOOL
FROM A LAYER IT ISN'T TESTING

Behavioural specs — Gherkin
WHAT the system does. Pure spec. Survives every swap.

Step definitions · page objects
HOW to drive it today. Rewritten when a driver changes.

Runners & drivers
Vitest · Playwright · Cypress — the same `.feature` file drives all of them.

SPREADING THE NET WIDE

a strategic distribution of test kinds — each catches what the others miss

`contract`	behaviour on every layer & adapter
`visual-diff`	what pixels look like, every theme
`e2e + smoke`	a thin layer over the real full stack
`25 gates`	architectural invariants, run first

Suites run in parallel — wall-clock is the slowest suite, not the sum.

// 02 · VERIFICATION

A bake-off, not redundancy.

SAME SPEC SURFACE —
ONE VARIABLE PER PEER

4 BROWSER PEERS

drive the real UI against in-process simulators

- Cucumber + Playwright
- native Playwright
- Cucumber + Cypress
- native Cypress

4 PRESENTER PEERS

drive the RxJS presenters in pure Node — no browser

- cucumber · real timers
- cucumber · fake timers
- vitest + quickpickle · fake
- vitest plain · fake

19x

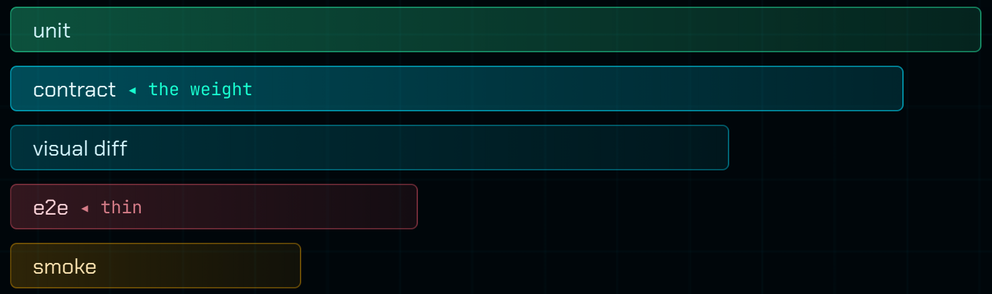
faster under virtual time — ~1s vs ~18.6s for the same 20 scenarios.

In production you'd pick one, well-informed. This conceptual project runs them side-by-side to see which wins in practice — the detail a paper POC always misses. AI makes that comparison cheap enough to actually do.

// 02 · THE RIGHT MIX

Lean on contracts. Distrust end-to-end.

PROPORTION, NOT DUPLICATION



E2E — THE ENEMY OF DETERMINISM

Flaky and slow. No fast feedback loop for an agent — a red you can't trust is worse than none. And usually only the happy path, so the gaps stay invisible.

CONTRACT — DETERMINISTIC, FAST, COMPREHENSIVE

One suite parameterised over **every adapter** — simulator and real socket — error paths included. The tight loop an agent needs to move fast.

// 02 · VERIFICATION

Goldens as the rendering contract.

TIER 1

`Playwright CT`

Component testing + the whole-app paint smoke.

TIER 2

`Plain Playwright`

Over a Vite host — URL-driven, verbatim-portable.

TIER 3

`Vitest browser`

`toMatchScreenshot` in real browser mode.

THREE TIERS = A BAKE-OFF · SHIP ONE, RUN IT HARD

The tiers compare visual-diff approaches in practice. In production you pick one — and push it aggressively, because the dumb UI makes near-total coverage actually attainable.

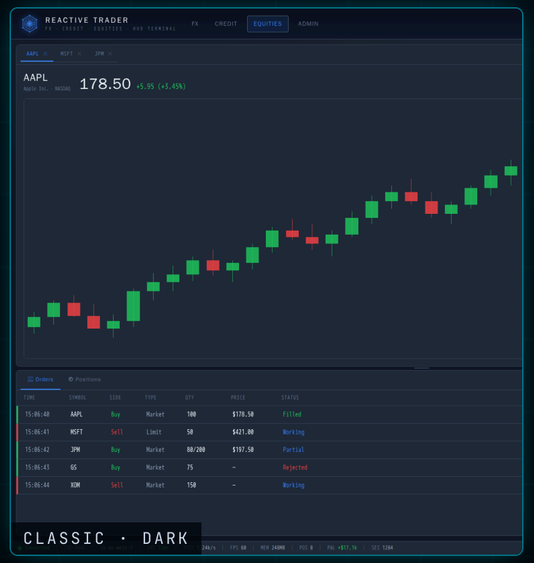
WHY 100% IS REAL HERE

Dumb components have `if`, never `while`. Finite branches, no loops — cover every branch and you've covered everything.

// 02 · VISUAL DIFF

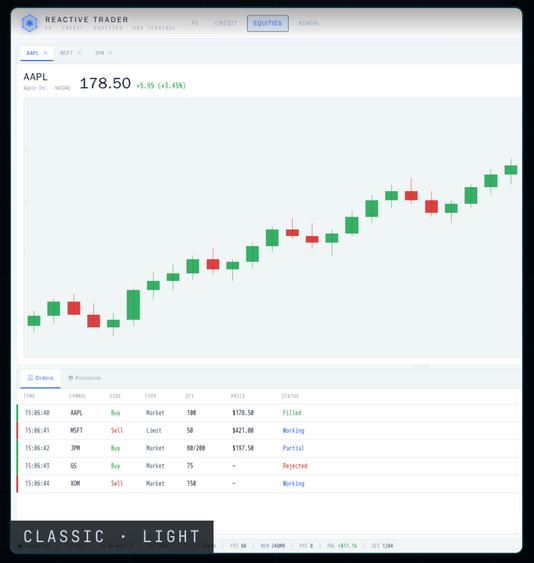
One page. Every theme × mode. Pixel-pinned.

REAL COMMITTED GOLDENS
app-equities · react/

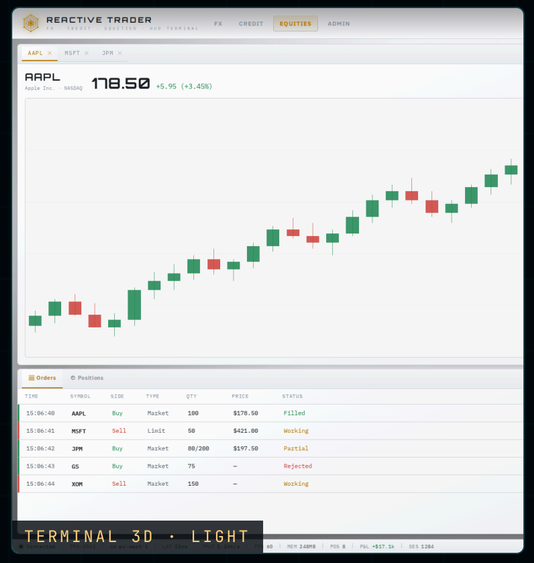












The dumb UI renders injected fakes behind one seam — no presenters, no domain, no server, no live streams, no timers. So we can snapshot whole pages, at any detail, across every skin and mode.

In a tightly-coupled legacy app you'd mock 40–50 dependencies to render one page deterministically — almost everyone gives up before starting.

DARK ROW · LIGHT ROW — SAME PAGE, EVERY SKIN

// 02 · VERIFICATION

The human's job: check the specs and the tests.

When the AI writes all the code, the code becomes cheap and disposable — the specs and the tests are the durable asset. So that's what we review, not every generated line. In that sense, the verification matters more than the code itself.

1 · CHECK THE SPECS

Are they precise, unambiguous, and exactly what we want built? Tight specs give the AI less room to drift. And the Cucumber peers consume those specs directly — the original experiment drifted partly because its tests came from the new code, not the specs.

2 · CHECK THE TESTS

Do they actually do what they claim, reflect the behaviour we want, and leave no gaps in coverage? Find a gap, backfill it. The tests now assert the spec's literal numbers and strings — not whatever the build happened to do.

03

THE PAYOFF

What it unlocked

A second platform. A framework regeneration. Devtools. And an AI that trades.
All of it — with the domain byte-identical.

// 03 · THE MULTI-CLIENT PROOF

A whole second platform. Zero domain changes.

THEORY → MEASUREMENT
REACT NATIVE / EXPO

REUSED VERBATIM

- ✓ `@rtc/client-core` — every presenter, machine, WsAdapter, port factory
- ✓ `@rtc/react-bindings` — the whole ViewModel
- ✓ `@rtc/domain` — entities, use cases, simulators

NEWLY WRITTEN (RING ④)

- dumb RN screens (react-native-svg charts)
- `buildNativePorts` + 2 platform adapters
- AsyncStorage / Appearance adapters
- token subset + expo-router tabs + AppRoot

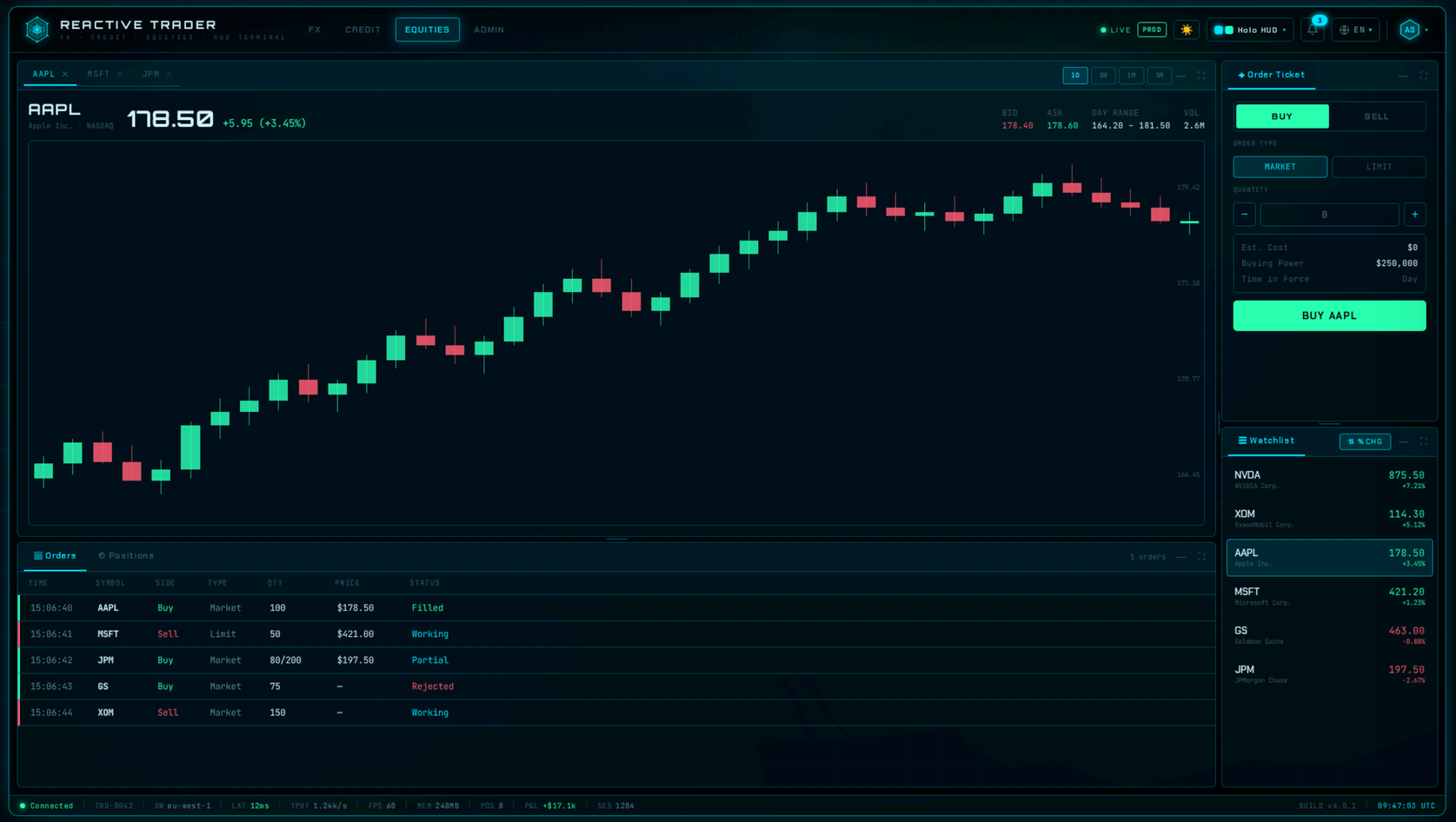
The same web `Tile` *and RN* `SpotTile` *invoke the same* `ExecuteTradeUseCase`.

// 03 · THE MULTI-CLIENT PROOF

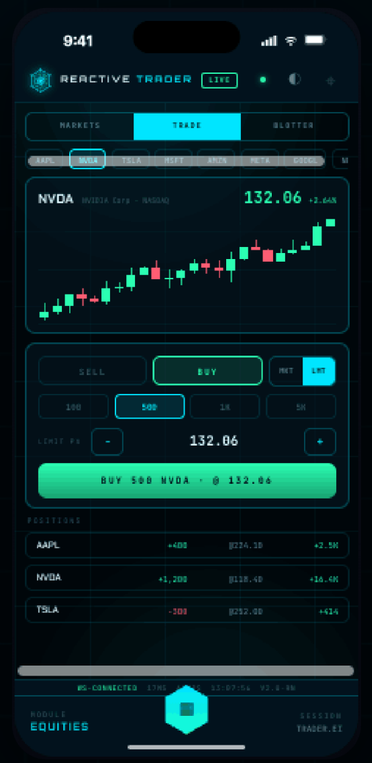
Same screen. Two clients. One core.

● RN UI · IN PROGRESS

WEB · REACT 19 + CSS MODULES



MOBILE · REACT NATIVE / EXPO



Different UI stack.
Different render model.
Identical core.

`@rtc/domain`
`@rtc/client-core`
`@rtc/react-bindings`

↑ shared verbatim

Still in progress: a working build already proves the reuse. What's left is the UI/UX polish — with barely a line of business logic to touch.

// 03 · REGENERATION · IN PROGRESS

Regenerating the client in SolidJS.

● ~50% COMPLETE

~1 day

`@rtc/solid-bindings`

StateObservable → Solid signal. The same primitive react-rxjs consumes.

~1 week

`@rtc/client-solid`

Rewrite dumb components. CSS Modules port verbatim.

Verification is already in place: framework-neutral contract specs, the Gherkin suites, and the visual goldens — the React golden is the pixel target.

THE REACT-SPECIFIC SURFACE

7 custom hooks total a Solid port must re-author

3 trivial context readers · 2 small pure helpers · only 2 of genuine complexity — and their FLIP math already lives in the framework-free `@rtc/motion-core`.

// 03 · REGENERATION · GOLDEN PARITY

One golden. Two frameworks must match it.

● SOLID FX · IN PROGRESS

@rtc/client-react · produces the golden



SAME
VISUAL-DIFF
SUITE

✓

PIXEL-FOR-PIXEL
OR CI IS RED

@rtc/client-solid · validated against it



Parity isn't eyeballed after the fact — it's **enforced by construction.** The target is frozen and shared before the Solid UI is even finished.

// 03 · PLANNED · FOR THE REDUX-DEVTOOLS CROWD

DevTools for a reactive graph.

THE OBSERVE STREAM

- machine states — every transition, explicit and inspectable
- live WebSocket traffic, framed and typed
- a `BroadcastChannel` bus — decoupled from the app
- time-scrub: replay the event log, rewind the UI

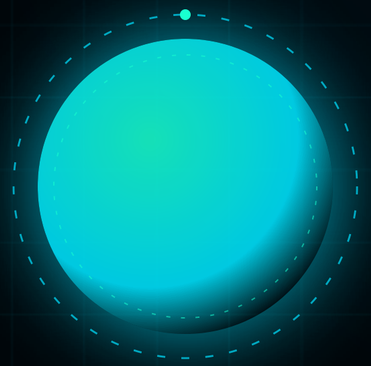
THE COMPOUNDING EFFECT

Because it's an event-shaped protocol over explicit machines, that same observe bus later becomes a tool source for Jarvis — an AI that debugs the running app it lives inside.

One workstream. Devtools now; a Jarvis capability for free.

// 03 · JARVIS · APPROVED DESIGN

AI as the third client.



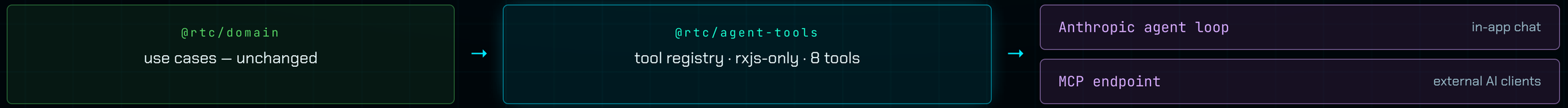
The web and RN clients already share the framework-free core. Jarvis is the third head — and it isn't even a UI framework. An AI agent consumes the application, not the DOM.

> *"None of it was designed for AI. Every AI-era capability falls out of decisions this repo enforced long before 'agent' was a product category. AI is simply the first non-UI consumer to arrive."*

The strongest test yet of the dependency rule.

// 03 · JARVIS · THE SURFACE

One registry. Two transports. Three consumers.



CONFIRM-GATED EXECUTION

`execute_trade` never runs on the model's say-so — the loop suspends on a confirm card with a 60s countdown that only a human action resolves.

MCP, IN-PROCESS

A colleague tells Claude Desktop "buy 2M GBP/USD" — the trade lands in the same live state the HUD is streaming, and appears live in the blotter.

// 03 · WHY IT'S A BOLT-ON, NOT A REWRITE

The seam was already there.

CAPABILITY	THE SEAM THAT MAKES IT A BOLT-ON	IN A fetch-in-useEffect CODEBASE
Agent tools over the domain	Use cases are plain classes over injected ports — callable from any process	Logic lives in onClick handlers; nothing callable to wrap
MCP server as a thin wrapper	Same registry, second transport — zero domain changes	The "API for the AI" is a parallel reimplementation, forever chasing the UI
Agent drives the UI	All UI state is machines with explicit intents	State scattered across useState islands; no surface to act on

The whole feature: one rxjs-only package, two adapters, one machine, one dumb panel — with `@rtc/domain` byte-identical.

04

FORWARD-LOOKING · NONE OF THIS IS BUILT YET

The singularity, on a seam.

Every capability below names an existing seam it plugs into. The ideas are designed against the architecture — not just remembered.

// 04 · TIER 1 · JAW-DROPPERS

The load-bearing primitive: the tool registry.

1 Generative UI bound to live streams

Jarvis emits a declarative panel spec; the client materialises a HUD panel wired to live ticks — it keeps updating after the model leaves.

`seam ▸ stream ports + a spec-interpreter machine`

2 Jarvis drives the app

"Set up my morning workspace" — tabs switch, panels rearrange, tiles glow as it narrates. The app visibly operating itself.

`seam ▸ UI state is already machines`

3 LLM market participants

Claude-driven counterparties that price, haggle, and hold grudges — multi-agent theatre inside the market itself.

`seam ▸ dealers are simulators behind ports`

4 Natural-language backtesting

"Would buying every EUR dip below 1.085 have paid today?" — a strategy run against recorded ticks, answered with an equity curve.

`seam ▸ price history + a sandboxed runner`

// 04 · TIERS 2 & 3

Deeper cuts — same load-bearing seams.

TIER 2 · ENGINEERING WOW

- The app introspects itself — an AI that debugs the app it lives in
- Time travel + explain — "what happened while I was away?"
- Sentinels as living HUD entities — standing watchers that flare on trigger
- Cross-device séance — start on web, continue on mobile

TIER 3 · ATMOSPHERE & STAGECRAFT

- Proactive narrator — deterministic triggers, LLM narrates
- Voice + wake word — "Jarvis— flatten my GBP exposure"
- War-game drills — "run a flash-crash drill"
- Ghost trader — a computer-use agent clicking the real UI

THE CAPABILITY-TO-SEAM MAP

Every capability attaches to a seam that already exists — or, for sentinels, the one new use case.

The seams are the investment. The capabilities are the interest.

05

THE MAP · STAYING IN CONTROL

Never surrender the driver's seat.

AI writes code faster than any human can read it — a metropolis of strangers, and real cognitive fatigue. The answer isn't more AI slop text. It's a map: docs and diagrams you can glance at to orient instantly. A picture is worth a thousand words — a film is worth hundreds of pictures.

// 05 · THE SYSTEM ON ONE SCREEN

Find your way out.



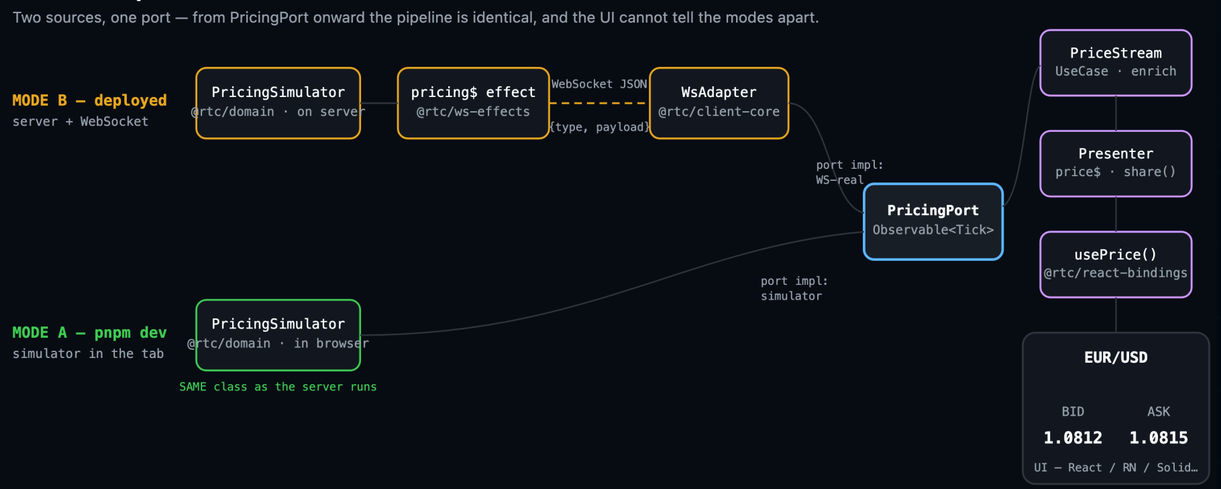
Two grand interchanges — the Domain and the UI — and one live network. Glance, orient, walk to the right station.

// 05 · MAPS THAT SHIP WITH THE CODE

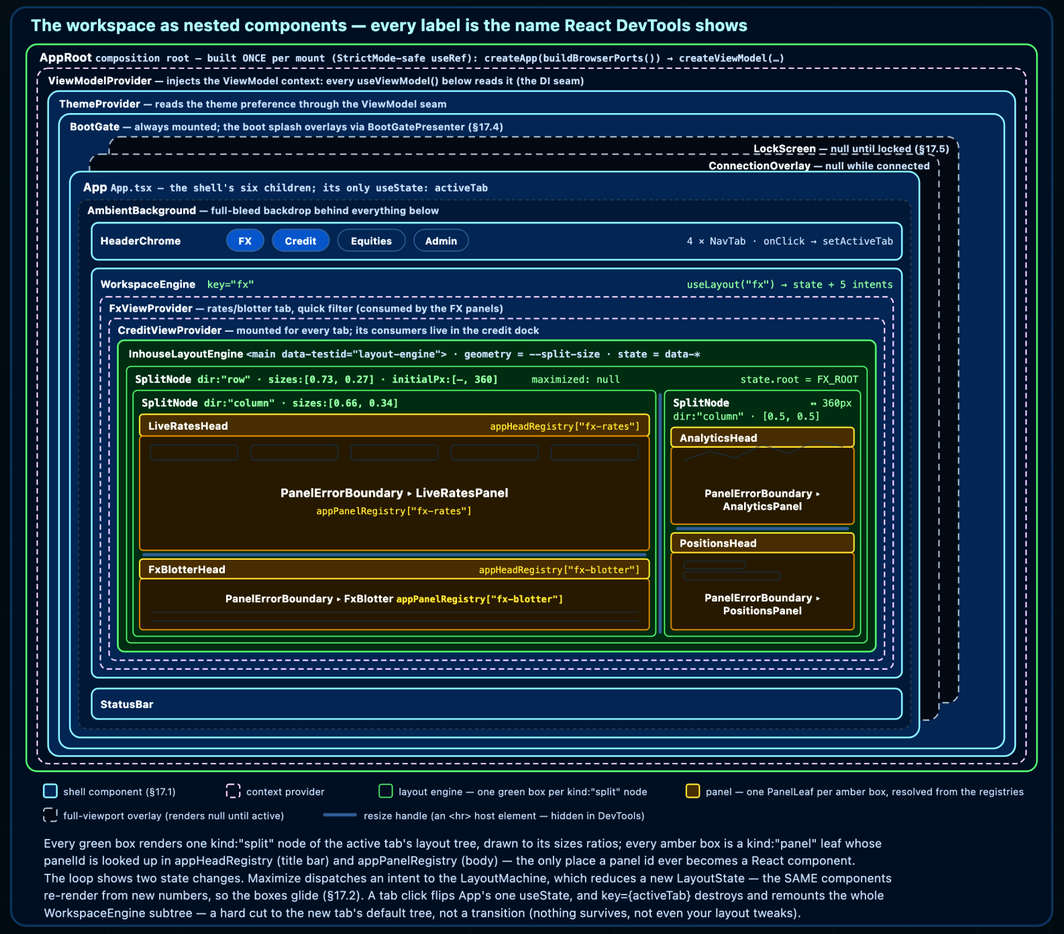
Not a metaphor — the actual diagrams in our docs.

A FILM > A PICTURE
FOR DYNAMIC BEHAVIOUR

tick-journey.svg · the life of a price tick



17-fx-devtools-boxes.svg · the workspace as components



Give someone a subway map and they navigate any city. Diagrams beside the code keep the human in the driving seat.



The seams are the investment. Engineering discipline builds them.

Clean boundaries make specs writable · specs make verification meaningful · meaningful verification lets an AI move fast without breaking things.

AND @rtc/domain NEVER CHANGED